

Index

- A. Java Fundamentals** (Questions 1 - 20)
- B. Variables, Operators and Control Statements** (Questions 21 - 36)
- C. Arrays and Strings** (Questions 37 - 55)
- D. OOP Concepts** (Questions 56 - 80)
- E. Access Modifiers, Packages and Core Concepts** (Questions 81 - 90)
- F. Exception Handling** (Questions 91 - 100)

A. Java Fundamentals

1. What is Java?

Java is a high-level, general-purpose, **object-oriented**, **class-based** programming language. It was developed by **James Gosling and his team at Sun Microsystems**, released in **1995**, and is now owned by **Oracle Corporation**.

Java programs are first compiled into platform-independent **bytecode**, which is then executed by the **Java Virtual Machine (JVM)**. This is why Java follows the slogan "**Write Once, Run Anywhere**" (**WORA**).

- **Uses:** web applications, Android apps, enterprise software, banking systems, desktop software, games, big-data tools.
- **Original name:** Oak, later renamed Java.

2. What are the main features of Java?

- **Simple:** easy syntax, similar to C/C++ but without pointers, operator overloading, or multiple inheritance of classes.
- **Object-oriented:** everything is organized into classes and objects.
- **Platform independent:** bytecode runs on any OS with a JVM.
- **Secure:** no explicit pointers, bytecode verifier, security manager, sandboxed execution.
- **Robust:** strong type checking, automatic garbage collection, and exception handling.
- **Multithreaded:** supports running several tasks concurrently using threads.
- **Portable:** the size of primitive types is fixed on all platforms.
- **High performance:** the JIT (Just-In-Time) compiler converts hot bytecode to native code.
- **Distributed:** supports networking (RMI, sockets, URLs).
- **Dynamic:** classes are loaded at runtime as needed.

3. Why is Java called platform independent?

Java is called platform independent because a Java program is compiled into **bytecode** (**.class** file) instead of machine code. Bytecode is not tied to any processor or operating system.

How it works:

- You write the program: **Hello.java** (source code).
- The compiler **javac** converts it into **Hello.class** (bytecode).
- Any machine with a **JVM** can run that same **.class** file. The JVM converts the bytecode to the native machine instructions of that particular system.

So, the **same bytecode runs on Windows, Linux, and macOS** without changes. Note that the **JVM is platform dependent** (a different JVM exists for each OS), but **Java programs are platform independent**.

4. What is the difference between source code, bytecode, and machine code?

Aspect	Source code	Bytecode	Machine code
Written/produced by	Programmer	Java compiler (javac)	JVM (interpreter / JIT)
File	.java	.class	Not stored (runtime) / native binary
Readable by humans	Yes	No (binary)	No (binary 0s and 1s)
Platform dependent	No	No	Yes (specific CPU/OS)
Executed by	Not executed directly	JVM	Processor (CPU)

Flow: Source code, then **javac**, then bytecode, then JVM, then machine code, then CPU.

5. What is the JVM?

The **Java Virtual Machine (JVM)** is an abstract (virtual) machine that provides the runtime environment in which Java bytecode is executed. It is part of the JRE.

Main responsibilities:

- **Loads** classes (Class Loader).
- **Verifies** bytecode for security and correctness (Bytecode Verifier).
- **Executes** bytecode (Interpreter + JIT compiler).
- **Manages memory** (Heap, Stack, Method Area) and performs **garbage collection**.

The JVM makes Java platform independent: Windows, Linux, and macOS each have their own JVM, but all run the same bytecode.

6. What is the JDK?

The **Java Development Kit (JDK)** is a software development kit used to **develop, compile, debug, and run** Java applications.

It contains:

- The **JRE** (so programs can run), and
- **Development tools:** `javac` (compiler), `java` (launcher), `jar` (archiver), `javadoc` (documentation generator), `jdb` (debugger), and others.

A developer needs the JDK. A user who only runs Java programs needs just the JRE.

7. What is the JRE?

The **Java Runtime Environment (JRE)** provides the minimum environment required to **run** Java applications. It is for end users who only want to run programs, not develop them.

It contains:

- The **JVM**, and
- The **core class libraries** (such as `java.lang`, `java.util`, `java.io`) and supporting files.

The JRE does **not** contain a compiler or developer tools. (Since Java 11, Oracle no longer ships a separate JRE download; the JDK includes what is needed.)

8. What is the difference between JDK, JRE, and JVM?

Feature	JDK	JRE	JVM
Full form	Java Development Kit	Java Runtime Environment	Java Virtual Machine
Purpose	Develop and run programs	Run programs	Execute bytecode
Contains	JRE + dev tools (<code>javac</code> , etc.)	JVM + libraries	Class loader, verifier, interpreter, JIT, GC
Used by	Developers	End users	Part of JRE (internal)
Platform dependent	Yes	Yes	Yes

Relationship: JDK = JRE + development tools, and JRE = JVM + libraries. So the JDK contains the JRE, which contains the JVM.

9. What is bytecode?

Bytecode is the **intermediate, platform-independent, highly optimized set of instructions** generated by the Java compiler from source code. It is stored in `.class` files.

- It is **not** human readable and **not** machine code for any real CPU.
- It is executed by the **JVM**, which interprets it or compiles it with the JIT compiler into native machine code.
- It is the reason for Java's portability and also helps security, since the JVM verifies bytecode before running it.

You can inspect it with the command `javap -c ClassName`.

10. Why is Java considered an object-oriented programming language?

Java organizes programs around **objects and classes**, and it supports all four main OOP principles:

- **Encapsulation:** data and methods bundled in classes, with access control.
- **Inheritance:** a class can extend another class (**extends**).
- **Polymorphism:** overloading and overriding.
- **Abstraction:** abstract classes and interfaces.

Almost everything (apart from primitive types) is an object, and all classes inherit from the **Object** class. Strictly speaking, Java is **not 100% pure** object-oriented, because it has **8 primitive data types** that are not objects (wrapper classes are provided to convert them).

11. What is the role of the main() method in Java?

main() is the **entry point** of a standalone Java application. When you run **java ClassName**, the JVM looks for this method and begins execution from its first statement. The program ends when **main()** finishes (and all non-daemon threads complete).

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

The **String[] args** parameter receives **command-line arguments** passed when the program is started.

12. Why is main() declared as public static void?

public static void main(String[] args)

- **public:** the JVM is outside the class, so the method must be accessible to it from anywhere. If it were private, the JVM could not call it.
- **static:** the JVM must call **main()** **without creating an object** of the class. Static methods belong to the class, not to objects.
- **void:** **main()** does not return any value to the JVM, since there is no one to receive it.
- **main:** the specific name the JVM searches for.
- **String[] args:** an array to receive command-line arguments.

13. Can we execute a Java program without a main() method?

In **modern Java**, **no**. If a class has no **main()** method, running it gives the error: **"Error: Main method not found in class ..."**.

- **Before Java 7:** you could put code in a **static** block, which runs when the class loads. The output was printed and then the JVM threw **NoSuchMethodError: main**.
- **Java 7 onward:** the JVM checks for **main()** first, so a program using only a static block fails with the error above.
- **Note:** in Java 21+ there is a preview feature (unnamed classes/instance main methods) that relaxes the signature, but a main-style entry point is still required.

(Also, applets and servlets run without **main()**, because a container/browser launches them.)

14. What is a Java class?

A **class** is a **blueprint or template** from which objects are created. It defines the **properties (fields/variables)** and **behaviors (methods)** common to all its objects. It is a logical entity.

```
class Student {  
    String name;           // field (state)  
    int age;  
    void display() {       // method (behavior)  
        System.out.println(name + " " + age);  
    }  
}
```

15. What is an object?

An **object** is a **real-world entity and an instance of a class**. It has three characteristics:

- **State:** the data stored in its fields (e.g., name, age).
- **Behavior:** what it can do, defined by methods.
- **Identity:** a unique address/hash that distinguishes it from other objects.

```
Student s1 = new Student();    // object creation  
s1.name = "Asha";  
s1.display();
```

Objects are created with the **new** keyword and are stored in **heap memory**.

16. What is the difference between a class and an object?

Class	Object
Blueprint/template	Instance of a class
Logical entity	Physical (real) entity
Declared once, using the class keyword	Created many times, using new
No memory allocated for instance data when declared	Memory allocated in the heap when created
Example: Car	Example: a specific red Honda car1

One class can be used to create any number of objects.

17. What is a reference variable?

A **reference variable** is a variable that stores the **memory address (reference)** of an object rather than the object itself. It is used to access the object's fields and methods.

```
Student s = new Student();    // 's' is the reference variable  
Student t = s;                // both s and t refer to the SAME object
```

- Reference variables live on the stack, while the objects they point to live on the heap.
- An uninitialized or cleared reference holds **null**; using it to access members throws **NullPointerException**.

18. What is the difference between primitive data types and reference data types?

Primitive	Reference
Store the actual value	Store the address of an object
Predefined by the language (8 types)	Created by the programmer or library (String , arrays, classes)
Fixed size	Size depends on the object
Cannot be null	Can be null
No methods	Have methods
Faster, stored directly (stack)	Object on heap, reference on stack
Name starts in lowercase (int)	Name usually starts with uppercase (String)

19. What are the eight primitive data types in Java?

Type	Size	Range / Values	Default
byte	1 byte	-128 to 127	0
short	2 bytes	-32,768 to 32,767	0
int	4 bytes	about -2.1 billion to 2.1 billion	0
long	8 bytes	about -9.2×10^{18} to 9.2×10^{18}	0L
float	4 bytes	single-precision decimal	0.0f
double	8 bytes	double-precision decimal	0.0d
char	2 bytes	a single Unicode character (0 to 65,535)	'\u0000'
boolean	JVM dependent	true or false	false

20. What is type casting?

Type casting is converting a value of one data type into another data type. It is of two kinds:

1. Widening (implicit / automatic): smaller type to larger type. No data loss, done automatically by the compiler.

byte -> short -> int -> long -> float -> double

```
int a = 10;
double d = a;    // 10.0
```

2. Narrowing (explicit / manual): larger type to smaller type. Possible data loss, must be done by the programmer using a cast operator.

```
double d = 9.78;
int x = (int) d;    // 9 (decimal part lost)
```

For objects, casting between parent and child types is called **upcasting** (automatic) and **downcasting** (explicit).

B. Variables, Operators and Control Statements

21. What is a local variable?

A **local variable** is declared **inside a method, constructor, or block**.

- Scope: only within that method/block.
- Created when the block is entered, destroyed when it exits (stored on the stack).

- **No default value:** it must be initialized before use, or the compiler reports an error.
- Cannot use access modifiers or **static**.

```
void show() {
    int x = 10;        // local variable
    System.out.println(x);
}
```

22. What is an instance variable?

An **instance variable** is declared **inside a class but outside any method, constructor, or block**, and without the **static** keyword.

- Each object gets its **own separate copy**.
- Created when the object is created, destroyed when the object is garbage collected (stored on the heap).
- Has a **default value** (0, false, null).
- Can have access modifiers.

```
class Student {
    String name;    // instance variable
    int age;        // instance variable
}
```

23. What is a static variable?

A **static variable** (class variable) is declared with the **static** keyword inside a class, outside methods.

- Only **one copy** exists, **shared by all objects** of the class.
- Memory is allocated once when the class is loaded.
- Accessed using the class name: **ClassName.variable**.
- Useful for common properties, e.g., a college name or a counter.

```
class Student {
    static String college = "ABC College";    // shared
    String name;                             // separate per object
}
```

24. What is the difference between == and =?

- **= (assignment operator):** assigns the value on the right to the variable on the left. **int x = 5;**
- **== (equality/relational operator):** compares two values and returns a boolean (**true/false**). **x == 5** gives **true**.

Using **=** instead of **==** inside a condition is a common bug (and a compile error in **if** for non-boolean types).

25. What is the difference between == and .equals()?

- **==** compares **values** for primitives, but for objects it compares **references (memory addresses)**, meaning whether both variables point to the same object.
- **.equals()** is a method that compares the **contents** (logical equality). Classes like **String** override it. For classes that do not override it, **Object.equals()** behaves like **==**.

```
String a = new String("hello");
String b = new String("hello");
System.out.println(a == b);        // false (different objects)
System.out.println(a.equals(b));    // true  (same content)
```

Note: **"hello" == "hello"** (two literals) is **true** because both refer to the same object in the String pool.

26. What are relational operators in Java?

Relational (comparison) operators compare two values and return a **boolean** result.

Operator	Meaning	Example (a=10, b=5)
<code>==</code>	equal to	<code>a == b</code> is false
<code>!=</code>	not equal to	<code>a != b</code> is true
<code>></code>	greater than	<code>a > b</code> is true
<code><</code>	less than	<code>a < b</code> is false
<code>>=</code>	greater than or equal	<code>a >= b</code> is true
<code><=</code>	less than or equal	<code>a <= b</code> is false

They are mostly used in **if** conditions and loops.

27. What are logical operators?

Logical operators combine or invert **boolean** expressions.

Operator	Name	Result
<code>&&</code>	Logical AND	true only if both are true
<code> </code>	Logical OR	true if at least one is true
<code>!</code>	Logical NOT	reverses the value

```
int age = 20;
if (age > 18 && age < 60) { /* adult */ }
```

`&&` and `||` are **short-circuit** operators: the right operand is skipped when the result is already known.

28. What is the difference between `&&` and `&`?

<code>&&</code> (logical AND)	<code>&</code> (bitwise / non-short-circuit AND)
Works only on booleans	Works on booleans and integers (bitwise)
Short-circuits: if the left side is false, the right side is NOT evaluated	Always evaluates both sides
Faster, safer	Used when side effects on both sides are needed

```
int x = 0;
if (x != 0 && 10 / x > 1) { } // safe: no division by zero
if (x != 0 & 10 / x > 1) { } // throws ArithmeticException
```

29. What is the ternary operator?

The **ternary operator** `? :` is the only Java operator with three operands. It is a compact form of if-else.

Syntax: `condition ? expressionIfTrue : expressionIfFalse;`

```
int a = 10, b = 20;
int max = (a > b) ? a : b; // 20
String r = (a % 2 == 0) ? "Even" : "Odd";
```

30. What is the difference between break and continue?

break	continue
Terminates the loop (or switch) completely	Skips the current iteration only
Control moves to the statement after the loop	Control goes to the next iteration (update/condition check)
Used in loops and switch	Used only in loops

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) break;        // prints 1 2
    System.out.print(i + " ");
}
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;    // prints 1 2 4 5
    System.out.print(i + " ");
}
```

31. What is the difference between while and do-while?

while	do-while
Entry-controlled: condition checked before the body	Exit-controlled: condition checked after the body
May execute zero times	Executes at least once
No semicolon after the condition	Semicolon required after while(condition);

```
int i = 10;
while (i < 5) { System.out.println(i); }    // no output
do { System.out.println(i); } while (i < 5); // prints 10 once
```

32. When would you use a for loop?

Use a **for loop** when the number of iterations is known in advance, for example counting from 1 to 10 or traversing every element of an array.

Syntax: **for** (initialization; condition; update) { body }

```
for (int i = 0; i < arr.length; i++) {
    System.out.println(arr[i]);
}
```

Java also has the **enhanced for loop** (for-each) for arrays and collections: **for** (int x : arr) { ... }. Use **while** when the number of iterations is unknown (e.g., reading until end of input).

33. What is a nested loop?

A **nested loop** is a loop placed **inside another loop**. For each single iteration of the outer loop, the inner loop runs completely.

```
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
// *
// * *
// * * *
```

Uses: printing patterns, processing 2D arrays/matrices, multiplication tables, comparing pairs of elements. If both loops run n times, the total iterations are n x n.

34. What is a switch statement?

A **switch statement** selects one block of code to execute from many alternatives by comparing a single expression with several constant **case** labels. It is a cleaner alternative to long if-else-if ladders.

```
int day = 2;
switch (day) {
    case 1: System.out.println("Mon"); break;
    case 2: System.out.println("Tue"); break;
    default: System.out.println("Other");
}
```

- **break** stops execution from falling into the next case.
- **default** runs if no case matches (optional).
- Case values must be **constants** and unique.

35. Can a switch statement work with a String?

Yes, since **Java 7**. The string in the switch is compared with the case labels using **equals()** (so it is case-sensitive).

```
String cmd = "start";
switch (cmd) {
    case "start": System.out.println("Starting"); break;
    case "stop": System.out.println("Stopping"); break;
    default: System.out.println("Unknown");
}
```

Switch supports **byte**, **short**, **int**, **char**, their wrappers, **String**, and **enum**. It does **not** support **long**, **float**, **double**, or **boolean**.

36. What happens if break is omitted from a switch case?

If **break** is omitted, **fall-through** occurs: execution continues into the next case(s), **even if their labels do not match**, until a **break** is found or the switch ends.

```
int x = 1;
switch (x) {
    case 1: System.out.println("One");
    case 2: System.out.println("Two");
    case 3: System.out.println("Three"); break;
}
// Output: One Two Three
```

Fall-through is sometimes used deliberately to group cases that share code.

C. Arrays and Strings

37. What is an array?

An **array** is a **container object that holds a fixed number of values of the same data type** in contiguous memory locations. Each element is accessed by its **index**, which starts from **0**.

```
int[] marks = {80, 75, 90};
System.out.println(marks[0]); // 80
```

- Arrays are **objects** in Java and are stored in the heap.
- Advantages: random access by index, simple storage of multiple values. Disadvantage: size cannot change.

38. Why are arrays called fixed-size data structures?

The **size of an array is decided when it is created** and **cannot be increased or decreased** afterward. The JVM allocates a contiguous block of memory of exactly that size.

```
int[] a = new int[3];    // size permanently 3
```

To "resize", you must create a new, larger array and copy the elements (e.g., with `Arrays.copyOf()`), or use a dynamic structure such as `ArrayList`.

39. How do you declare and initialize an array in Java?

Declaration:

```
int[] arr;           // preferred style
int arr2[];          // also valid
```

Creation (with size): `arr = new int[5];`

Initialization with values:

```
int[] a = {10, 20, 30};           // shorthand
int[] b = new int[]{1, 2, 3};      // with new
int[] c = new int[3];              // default values (0)
c[0] = 5;                          // assign by index
```

40. What is the default value of elements in an integer array?

0. Array elements are automatically initialized with default values:

Type	Default
<code>int</code> , <code>short</code> , <code>byte</code> , <code>long</code>	0
<code>float</code> , <code>double</code>	0.0
<code>boolean</code>	false
<code>char</code>	'\u0000' (null character)
Objects (e.g., <code>String</code>)	null

```
int[] a = new int[3];
System.out.println(a[0]);    // 0
```

41. How do you find the length of an array?

Use the **length property** (a field, not a method, so **no parentheses**):

```
int[] a = {5, 6, 7, 8};
System.out.println(a.length);    // 4
```

Do not confuse this with `String.length()` (a method) or `ArrayList.size()`. For a 2D array, `m.length` is the number of rows and `m[0].length` is the number of columns in row 0.

42. What happens when you access an invalid array index?

Java throws an `ArrayIndexOutOfBoundsException` at **runtime**. Valid indexes run from **0 to length-1**. A negative index or an index $\geq$ length is invalid.

```
int[] a = new int[3];
a[3] = 10;    // Exception: Index 3 out of bounds for length 3
```

It is an unchecked exception. If not handled, the program terminates. Always check bounds (`i < a.length`) or use try-catch.

43. What is a two-dimensional array?

A **two-dimensional array** is an **array of arrays**, which is used to represent data in rows and columns (a table or matrix).

```
int[][] m = new int[2][3];           // 2 rows, 3 columns
int[][] n = {{1, 2, 3}, {4, 5, 6}}; // with values
System.out.println(n[1][2]);         // 6
for (int i = 0; i < n.length; i++)
    for (int j = 0; j < n[i].length; j++)
        System.out.print(n[i][j] + " ");
```

Java also allows **jagged arrays**, where each row has a different length, e.g., `int[][] j = new int[3][];`.

44. What is the difference between an array and an ArrayList?

Array	ArrayList
Fixed size	Dynamic size (grows and shrinks)
Can store primitives and objects	Stores objects only (uses wrappers like <code>Integer</code>)
Length via <code>arr.length</code>	Size via <code>list.size()</code>
Part of the language	Part of the Collections framework (<code>java.util</code>)
No built-in methods	Has methods: <code>add()</code> , <code>remove()</code> , <code>contains()</code> , <code>get()</code>
Slightly faster	Slightly more overhead
Supports multi-dimensions	Single dimension (but a list of lists is possible)

45. What is a String in Java?

A **String** is a **sequence of characters** and is represented by the `java.lang.String` class. Strings are **objects**, not primitive types, and are **immutable**.

```
String s1 = "Hello";           // string literal (String Constant Pool)
String s2 = new String("Hello"); // new object in heap
```

- Literals with the same content share one object in the **String Constant Pool**, which saves memory.
- Provides many methods: `length()`, `charAt()`, `substring()`, `equals()`, `contains()`, etc.

46. Why is String immutable?

Immutable means once a String object is created, its content **cannot be changed**. Operations such as `concat()` or `toUpperCase()` create a **new** String.

Reasons:

- **Security:** Strings hold sensitive data (passwords, URLs, file paths, class names). If they could change, security could be broken.
- **String pool:** many references can safely share one object, which saves memory.
- **Thread safety:** immutable objects can be shared across threads without synchronization.
- **HashCode caching:** the hashcode is calculated once and cached, which makes Strings efficient keys in `HashMap`.

```
String s = "Java";
s.concat(" World");           // result ignored
System.out.println(s);        // still "Java"
```

47. What is the difference between String, StringBuilder, and StringBuffer?

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Speed	Slow for repeated changes	Fastest	Slower than StringBuilder
Introduced	Java 1.0	Java 1.5	Java 1.0
Use when	Content does not change	Many changes, single thread	Many changes, multiple threads

```
StringBuilder sb = new StringBuilder("Hi");
sb.append(" there");    // same object modified
```

48. What is the difference between equals() and equalsIgnoreCase()?

Both compare the **content** of two strings and return a boolean.

- **equals()** is **case-sensitive**.
- **equalsIgnoreCase()** **ignores case** differences.

```
String a = "Java", b = "JAVA";
System.out.println(a.equals(b));           // false
System.out.println(a.equalsIgnoreCase(b)); // true
```

49. What is the use of length() in a String?

length() returns the **number of characters** in the string (including spaces) as an **int**.

```
String s = "Hello World";
System.out.println(s.length());    // 11
```

It is a **method** for Strings (needs parentheses), unlike **array.length**, which is a field.

50. What is the use of substring()?

substring() **extracts a part of a string** and returns a new String.

- **substring(beginIndex)**: from **beginIndex** to the end.
- **substring(beginIndex, endIndex)**: from **beginIndex** up to **endIndex - 1** (the end index is exclusive).

```
String s = "Hello World";
s.substring(6);           // "World"
s.substring(0, 5);        // "Hello"
```

An invalid index throws **StringIndexOutOfBoundsException**.

51. What is the use of charAt()?

charAt(index) returns the **character at the specified index** (0-based).

```
String s = "Java";
System.out.println(s.charAt(0));    // J
System.out.println(s.charAt(3));    // a
```

If the index is negative or $\geq$ **length()**, it throws **StringIndexOutOfBoundsException**. It is commonly used to loop through each character of a string.

52. What is the difference between trim() and strip()?

Both remove **leading and trailing white space** and return a new string.

- `trim()` (old, Java 1.0) removes characters with code `<= U+0020` (basic ASCII spaces, tabs, newlines).
- `strip()` (Java 11+) removes all white space as defined by `Character.isWhitespace()`, so it is **Unicode-aware** (e.g., it removes the Unicode em-space).
- Java 11 also adds `stripLeading()` and `stripTrailing()`.

```
String s = "  Hello  ";
s.trim();    // "Hello"
s.strip();   // "Hello"
```

53. What do toUpperCase() and toLowerCase() do?

- `toUpperCase()` converts all characters to **upper case**.
- `toLowerCase()` converts all characters to **lower case**.

They return a **new String** (the original is unchanged because Strings are immutable).

```
String s = "Java";
System.out.println(s.toUpperCase()); // JAVA
System.out.println(s.toLowerCase()); // java
```

Often used for case-insensitive comparisons and normalizing input.

54. What is the use of contains() in String?

`contains(CharSequence s)` checks whether the string **contains the given sequence of characters**. It returns `true` or `false` and is **case-sensitive**.

```
String s = "Welcome to Java";
System.out.println(s.contains("Java")); // true
System.out.println(s.contains("java")); // false
```

Passing `null` throws `NullPointerException`. (Related: `indexOf()`, `startsWith()`, `endsWith()`.)

55. What is the difference between concat() and the + operator for Strings?

<code>concat()</code>	<code>+</code> operator
A method of <code>String</code>	An operator (overloaded for strings)
Accepts only a String argument	Accepts any type (auto-converted to string)
Throws <code>NullPointerException</code> if the argument is <code>null</code>	Treats <code>null</code> as the text "null"
Joins only two strings at a time	Can join many values in one expression
Returns the same string if the argument is empty	Always produces a new string

```
"Java".concat(" Fun"); // "Java Fun"
"Age: " + 25;          // "Age: 25"
```

D. OOP Concepts

56. What are the four main principles of OOP?

- **Encapsulation:** binding data and methods together and hiding data using access control.
- **Inheritance:** a class acquiring properties and behavior of another class.
- **Polymorphism:** one name, many forms (overloading and overriding).
- **Abstraction:** hiding implementation details and showing only essential features (abstract classes and interfaces).

Together they make code reusable, secure, flexible, and easier to maintain.

57. What is encapsulation?

Encapsulation is the process of **wrapping data (variables) and the code that operates on it (methods) into a single unit (class) and restricting direct access** to the data. It is also called **data hiding**.

Think of a medicine capsule: the contents are protected inside.

Benefits: data security, control over how data is changed (validation), flexibility to change internal implementation, easier maintenance, and read-only or write-only fields are possible.

58. How is encapsulation achieved in Java?

Steps:

- Declare the class variables as **private**.
- Provide **public getter and setter methods** to read and modify them.

```
class Account {  
    private double balance;           // hidden  
  
    public double getBalance() { return balance; }  
  
    public void setBalance(double b) {  
        if (b >= 0) balance = b;      // validation  
    }  
}
```

Outside code cannot directly write `acc.balance = -500;` it must use the setter, which can validate the value.

59. What is inheritance?

Inheritance is the mechanism by which one class (the **subclass / child / derived class**) **acquires the fields and methods of another class** (the **superclass / parent / base class**) using the **extends** keyword. It represents an **IS-A relationship** (a Dog IS-A Animal).

```
class Animal { void eat() { System.out.println("eating"); } }  
class Dog extends Animal { void bark() { System.out.println("barking"); } }  
  
Dog d = new Dog();  
d.eat();    // inherited  
d.bark();
```

Types in Java (classes): single, multilevel, hierarchical. **Multiple inheritance of classes is not supported** (to avoid the diamond problem), but it is achieved through interfaces.

60. Why is inheritance used?

- **Code reusability:** write common code once in the parent class and reuse it in children.
- **Method overriding:** enables runtime polymorphism.
- **Extensibility:** new classes can be built on existing ones without modifying them.
- **Logical organization:** creates a natural hierarchy (Vehicle, Car, Bike).
- **Easier maintenance:** a change in the parent applies to all children.

61. What is the difference between extends and implements?

extends	implements
Used by a class to inherit another class	Used by a class to implement an interface
Also used by an interface to extend another interface	Cannot be used by an interface
A class can extend only one class	A class can implement multiple interfaces
Child may or may not override parent methods	Class must implement all abstract methods of the interface

```
class B extends A implements I1, I2 { }
```

62. What is polymorphism?

Polymorphism means "**many forms**": the ability of a single method, operator, or object to behave differently in different situations.

Types:

- **Compile-time (static) polymorphism:** method overloading.
- **Runtime (dynamic) polymorphism:** method overriding.

Real-life example: a person is a student, a son, and a friend at the same time, behaving differently in each role. In Java, `print(int)` and `print(String)` are the same name with different behavior.

63. What is compile-time polymorphism?

Compile-time (static) polymorphism is when the compiler decides **which method to call at compile time** (static/early binding). It is achieved through **method overloading** (and operator overloading for `+` with Strings).

```
class Calc {
    int add(int a, int b)          { return a + b; }
    double add(double a, double b){ return a + b; }
    int add(int a, int b, int c){ return a + b + c; }
}
```

The compiler picks the version by looking at the number and types of the arguments.

64. What is runtime polymorphism?

Runtime (dynamic) polymorphism is when **which method runs is decided at runtime** based on the **actual object**, not the reference type (dynamic/late binding). It is achieved through **method overriding** together with **upcasting** (a parent reference pointing to a child object).

```
class Animal { void sound() { System.out.println("Some sound"); } }
class Dog extends Animal { void sound() { System.out.println("Bark"); } }
class Cat extends Animal { void sound() { System.out.println("Meow"); } }

Animal a = new Dog();
a.sound();    // Bark
a = new Cat();
a.sound();    // Meow
```

Note: variables and static/private/final methods are not resolved at runtime.

65. What is method overloading?

Method overloading is defining **multiple methods with the same name in the same class but with different parameter lists** (different number of parameters, different types, or different order of types).

```
void show(int a) { }
void show(String s) { }
void show(int a, String s) { }
void show(String s, int a) { }
```

- It is **compile-time polymorphism**.
- It increases readability since one name is used for related actions.
- Overloading is **not** possible by changing only the return type.

66. What is method overriding?

Method overriding is when a **subclass provides its own implementation of a method that is already defined in its superclass**. It requires an IS-A (inheritance) relationship.

Rules:

- Same method name and same parameters.
- Return type must be the same or a subtype (covariant).
- Access level cannot be more restrictive than the parent's.
- **static**, **private**, and **final** methods cannot be overridden.
- Use the **@Override** annotation (recommended).

```
class Bank { double rate() { return 5.0; } }
class SBI extends Bank {
    @Override
    double rate() { return 6.5; }
}
```

67. What is the difference between method overloading and overriding?

Overloading	Overriding
Same name, different parameters	Same name, same parameters
In the same class (or subclass)	Needs inheritance (parent and child)
Compile-time polymorphism	Runtime polymorphism
Return type may differ	Return type same or covariant
Access modifier can be anything	Cannot reduce visibility
Static and private methods can be overloaded	Static, private, final methods cannot be overridden
Improves readability	Provides specific implementation of inherited method

68. Can we overload a method by changing only its return type?

No. The compiler gives a **compile-time error** (*"method already defined"*). Overloading is decided only from the **method signature** (name + parameter list), and the return type is not part of the signature.

```
int add(int a, int b)    { return a + b; }
double add(int a, int b) { return a + b; } // ERROR
```

If it were allowed, a call like **add(2, 3)** would be ambiguous, since the compiler could not tell which version you intend.

69. What is the super keyword?

super is a **reference to the immediate parent class object**.

Uses:

- **Call parent method:** **super.display();**

- **Access parent field** (when hidden by the child's field): `super.name`
- **Call parent constructor:** `super(args);` which **must be the first statement** in the child constructor. If not written, the compiler inserts `super();` automatically.

```
class A { int x = 10; void show() { System.out.println("A"); } }
class B extends A {
    int x = 20;
    void show() {
        super.show();           // A
        System.out.println(super.x); // 10
    }
}
```

70. What is the this keyword?

`this` is a **reference to the current object** (the object whose method or constructor is being executed).

Uses:

- **Distinguish instance variables from parameters** with the same name.
- **Call another constructor** of the same class: `this(args);` (must be the first statement).
- **Call the current object's method:** `this.show();`
- **Pass the current object** as an argument or **return** it (method chaining).

```
class Student {
    String name;
    Student(String name) {
        this.name = name;    // field = parameter
    }
}
```

`this` cannot be used in a static context.

71. What is a constructor?

A **constructor** is a special block of code used to **initialize an object** at the moment it is created.

Characteristics:

- Has the **same name as the class**.
- Has **no return type** (not even void).
- Called **automatically** when an object is created with `new`.
- If the programmer writes none, the compiler supplies a **default (no-arg) constructor**.

Types: default, no-argument, and parameterized.

```
class Student {
    String name;
    Student(String n) { name = n; }    // parameterized constructor
}
Student s = new Student("Asha");
```

72. What is the difference between a constructor and a method?

Constructor	Method
Initializes an object	Performs an action/behavior
Name same as the class	Any valid name
No return type	Must have a return type (or void)
Called automatically on new	Called explicitly by the programmer
Called once per object creation	Can be called any number of times
Compiler provides a default one	Compiler does not provide methods
Cannot be static , final , abstract	Can be
Not inherited	Inherited by subclasses

73. Can a constructor be overloaded?

Yes. A class can have **multiple constructors with different parameter lists**. This is called **constructor overloading**, and it lets you create objects in different ways. One constructor can call another with **this(...)**.

```
class Student {
    String name; int age;
    Student() { name = "Unknown"; }
    Student(String n) { name = n; }
    Student(String n, int a) { this(n); age = a; }
}
```

74. Can a constructor be inherited?

No. Constructors are **not inherited** by subclasses, because they have the class's own name and are not members that can be passed down. However, a subclass constructor **can call** the parent constructor using **super()**, and the parent constructor always runs first when a child object is created.

```
class A { A() { System.out.println("A"); } }
class B extends A { B() { System.out.println("B"); } }
new B(); // Output: A B
```

75. What is an abstract class?

An **abstract class** is a class declared with the keyword **abstract**. It represents a **partially complete** concept, and **objects cannot be created** from it.

- May contain **abstract methods** (declared without a body) and **concrete methods** (with a body).
- Can have constructors, instance variables, static methods, and final methods.
- A subclass must **implement all abstract methods**, or itself be declared abstract.

```
abstract class Shape {
    abstract double area(); // no body
    void display() { System.out.println("Shape"); }
}
class Circle extends Shape {
    double area() { return 3.14 * 2 * 2; }
}
```

76. What is an interface?

An **interface** is a **reference type that defines a contract**: a set of methods that a class must implement. It is used to achieve **abstraction and multiple inheritance**.

- Declared with the **interface** keyword; implemented with **implements**.
- Methods are **public abstract** by default; variables are **public static final** (constants).
- Cannot be instantiated and has no constructors.

- From **Java 8**: **default** and **static** methods are allowed; from **Java 9**: **private** methods.

```
interface Drawable {
    void draw();
}
class Circle implements Drawable {
    public void draw() { System.out.println("Drawing circle"); }
}
```

77. Why do we use interfaces?

- Abstraction**: define what a class must do, without saying how.
- Multiple inheritance**: a class can implement many interfaces.
- Loose coupling**: code depends on the interface, not the implementation, so implementations can be swapped easily.
- Common standard**: unrelated classes can follow the same contract (e.g., **Comparable**, **Runnable**).
- Polymorphism**: an interface reference can hold any implementing object.

78. What is the difference between an abstract class and an interface?

Abstract class	Interface
Declared with abstract class	Declared with interface
Extended using extends	Implemented using implements
A class can extend only one	A class can implement many
Can have abstract and concrete methods	Abstract methods (plus default/static/private since Java 8/9)
Can have constructors	No constructors
Instance variables of any kind	Only public static final constants
Methods can have any access modifier	Methods are public
Use for related classes sharing code (IS-A)	Use for a capability/contract across unrelated classes

79. Can a class implement multiple interfaces?

Yes. A class can implement any number of interfaces, separated by commas. This is how Java achieves **multiple inheritance of type** without the diamond problem. The class must implement **all** abstract methods of all the interfaces.

```
interface Printable { void print(); }
interface Showable { void show(); }

class Doc implements Printable, Showable {
    public void print() { System.out.println("print"); }
    public void show() { System.out.println("show"); }
}
```

If two interfaces have a **default** method with the same signature, the class must override it to resolve the conflict.

80. Can an interface extend another interface?

Yes. An interface can **extend one or more** interfaces using **extends** (not **implements**). The child interface inherits all the abstract methods of its parents, so a class implementing it must define all of them.

```
interface A { void a(); }
interface B { void b(); }
interface C extends A, B { void c(); }

class Impl implements C {
    public void a() { }
    public void b() { }
    public void c() { }
}
```

An interface can extend multiple interfaces, while a class can extend only one class.

E. Access Modifiers, Packages and Core Concepts

81. What are the access modifiers in Java?

Access modifiers control the **visibility (accessibility)** of classes, variables, methods, and constructors. Java has four levels:

- **private**: accessible only within the same class.
- **default (no keyword)**: accessible within the same package.
- **protected**: accessible within the same package and in subclasses (even in other packages).
- **public**: accessible from everywhere.

Top-level classes can only be **public** or default.

82. What is the difference between private, default, protected, and public?

Modifier	Same class	Same package	Subclass (different package)	Different package (non-subclass)
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

Order from most restrictive to least: private, default, protected, public.

83. Why are data members generally declared private?

Because of **encapsulation (data hiding)**:

- **Protection**: outside code cannot change data directly or accidentally.
- **Validation**: setters can check values before accepting them (e.g., age cannot be negative).
- **Flexibility**: the internal implementation can change without affecting code that uses the class.
- **Control**: you can make fields read-only (getter only) or write-only.
- **Maintainability**: fewer places in the program depend on the class's internals, which reduces bugs.

84. What is a package?

A **package** is a **namespace that groups related classes, interfaces, enums, and sub-packages**, like a folder in a file system. It is declared with the **package** keyword as the first statement in the file.

```
package com.college.student;

public class Student { }
```

Types: built-in packages (**java.lang**, **java.util**, **java.io**, **java.sql**) and user-defined packages.

85. Why are packages used in Java?

- **Organization:** group related classes together.
- **Avoid name conflicts:** two classes with the same name can exist in different packages.
- **Access control:** package-private (default) and **protected** access depend on packages.
- **Reusability:** classes can be imported and reused in other programs.
- **Easier maintenance** and searching of large projects.
- **Encapsulation:** hide internal classes from the outside.

86. What is the difference between import and package?

package	import
Declares the package that the current class belongs to	Brings classes from other packages into the current file
Must be the first statement	Comes after the package statement
Only one per file	Can have many
Creates/defines a namespace	Lets you use a class without its full name

```
package myapp;           // this class belongs to 'myapp'
import java.util.Scanner; // use Scanner without java.util. prefix
import java.util.*;       // import all classes of the package
```

Note: **java.lang** is imported automatically.

87. What is the purpose of the static keyword?

static makes a member **belong to the class rather than to any object**.

Can be applied to: variables, methods, blocks, and nested classes.

- **Static variable:** one shared copy for all objects (saves memory).
- **Static method:** can be called with the class name without creating an object (e.g., **Math.sqrt()**); can access only static members directly; cannot use **this** or **super**.
- **Static block:** runs once when the class is loaded; used to initialize static data.
- **main()** is static so the JVM can call it without an object.

```
class Counter {
    static int count = 0;
    static void show() { System.out.println(count); }
}
Counter.show();
```

88. What is the purpose of the final keyword?

final is used to **restrict modification**. It can be applied to:

- **Variable:** its value cannot be changed after initialization (a constant): **final double PI = 3.14;**
- **Method:** cannot be **overridden** by subclasses.
- **Class:** cannot be **inherited** (e.g., **String**, **Integer**).
- **Parameter:** cannot be reassigned within the method.

```
final class A { }           // cannot be extended
class B { final void f() { } } // cannot be overridden
```

Not to be confused with **finally** (exception handling block) or **finalize()** (a method called by the garbage collector).

89. Can a final variable be modified?

No, a final variable **cannot be reassigned** once it has been given a value. Trying to do so is a compile-time error.

```
final int x = 10;
x = 20;      // ERROR
```

- A final variable must be initialized: at declaration, in a constructor (for instance fields), or in a static block (for static fields).
- For a **final reference variable**, the **reference cannot change**, but the **object's contents can**:

```
final StringBuilder sb = new StringBuilder("Hi");
sb.append(" there");    // allowed
sb = new StringBuilder(); // ERROR
```

90. Can a final method be overridden?

No. A method declared **final** **cannot be overridden** in any subclass. It can still be **inherited and called** by subclasses. Use it when you want to guarantee that a method's behavior stays the same.

```
class A {
    final void show() { System.out.println("A"); }
}
class B extends A {
    void show() { }    // ERROR: cannot override final method
}
```

A final method can be **overloaded**, though.

F. Exception Handling

91. What is an exception?

An **exception** is an **unwanted or unexpected event that occurs during program execution (runtime) and disrupts the normal flow** of instructions. In Java, an exception is an **object** created when an error occurs, belonging to a subclass of `java.lang.Throwable`.

Examples: dividing by zero (`ArithmeticException`), accessing an invalid array index (`ArrayIndexOutOfBoundsException`), using a null reference (`NullPointerException`), opening a missing file (`FileNotFoundException`), invalid number format (`NumberFormatException`).

92. What is exception handling?

Exception handling is the mechanism of **detecting, catching, and responding to runtime errors** so that the **normal flow of the program is maintained** and the program does not crash abruptly.

Java provides five keywords: **try**, **catch**, **finally**, **throw**, **throws**.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Done");
}
```

93. What is the difference between an error and an exception?

Error	Exception
Serious problem, usually related to the JVM or system resources	Problem in the program/code or environment
Cannot (and should not) be handled by the program	Can be handled using try-catch
Examples: <code>OutOfMemoryError</code> , <code>StackOverflowError</code> , <code>VirtualMachineError</code>	Examples: <code>IOException</code> , <code>ArithmeticException</code> , <code>NullPointerException</code>
All are unchecked	Can be checked or unchecked
Subclass of <code>java.lang.Error</code>	Subclass of <code>java.lang.Exception</code>

Both are subclasses of `Throwable`.

94. What is the difference between checked and unchecked exceptions?

Checked exception	Unchecked exception
Checked at compile time	Occur at runtime (not checked by compiler)
Must be handled (try-catch) or declared (throws) , otherwise a compile error	Handling is optional
Subclasses of <code>Exception</code> excluding <code>RuntimeException</code>	Subclasses of <code>RuntimeException</code> (and <code>Error</code>)
Usually caused by external factors (files, network, DB)	Usually caused by programming mistakes
Examples: <code>IOException</code> , <code>SQLException</code> , <code>FileNotFoundException</code> , <code>ClassNotFoundException</code>	Examples: <code>NullPointerException</code> , <code>ArithmeticException</code> , <code>ArrayIndexOutOfBoundsException</code> , <code>NumberFormatException</code>

95. What is the purpose of try and catch?

- **try block:** contains the code that **might throw an exception**. It must be followed by at least one **catch** or a **finally**.
- **catch block:** contains the code that **handles** a specific type of exception. It runs only if that exception occurs in the try block.

```
try {
    int[] a = new int[3];
    a[5] = 10;                // exception here
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Invalid index");
}
System.out.println("Program continues");
```

- You can use **multiple catch blocks** (more specific exceptions first, general ones later) or a multi-catch: `catch (A | B e)`.
- Statements after the exception point inside **try** are skipped.

96. What is the purpose of the finally block?

The **finally** block contains code that **always executes**, whether or not an exception occurs and whether or not it is caught. It is used for **cleanup**, such as closing files, database connections, or releasing resources.

```
try {
    // risky code
} catch (Exception e) {
    // handle
} finally {
    System.out.println("Always runs");
}
```

- It runs even if there is a **return** in **try** or **catch**.
- It does **not** run only when the JVM exits (**System.exit()**) or crashes.
- Modern alternative: **try-with-resources**, which closes resources automatically.

97. What is the difference between throw and throws?

throw	throws
Keyword used to explicitly throw an exception object	Keyword used in a method signature to declare the exceptions it might throw
Used inside a method/block	Used with the method declaration
Followed by an object : throw new Exception("msg");	Followed by class name(s) : throws IOException, SQLException
Can throw only one exception at a time	Can declare multiple exceptions
Used to raise an exception	Used to pass responsibility to the caller

```
void check(int age) throws Exception {
    if (age < 18)
        throw new Exception("Not eligible");
}
```

98. Can we create our own exception class?

Yes. This is called a **user-defined (custom) exception**. Create a class that **extends** **Exception** (checked) or **RuntimeException** (unchecked), and pass a message to the parent using **super(message)**. Then use **throw** to raise it.

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) { super(msg); }
}

class Test {
    static void validate(int age) throws InvalidAgeException {
        if (age < 18)
            throw new InvalidAgeException("Age must be 18 or above");
    }
    public static void main(String[] args) {
        try { validate(15); }
        catch (InvalidAgeException e) { System.out.println(e.getMessage()); }
    }
}
```

Custom exceptions help represent business rules clearly.

99. What happens if an exception is not handled?

If an exception is **not caught**, it **propagates up the call stack** from the method where it occurred to its caller, and so on up to `main()`. If no method handles it, the **JVM's default exception handler** takes over. It:

- prints the **exception name, message, and stack trace** to the console, and
- **terminates the program abnormally** (the remaining statements are not executed).

```
public static void main(String[] args) {  
    int x = 10 / 0; // unhandled  
    System.out.println("Not printed");  
}  
// Exception in thread "main" java.lang.ArithmeticException: / by zero
```

100. Why should exceptions be handled instead of allowing the program to terminate unexpectedly?

- **Program continuity:** the rest of the program can continue to run normally.
- **User-friendly:** show a meaningful message instead of a confusing stack trace.
- **Prevents data loss/corruption:** unsaved data and open transactions can be handled safely.
- **Resource cleanup:** files, connections, and memory can be released properly using `finally`.
- **Separation of concerns:** error-handling code is kept apart from normal logic, making code cleaner.
- **Debugging and logging:** errors can be recorded for diagnosis.
- **Reliability and professionalism:** robust applications do not crash on bad input or unexpected conditions.